

TDD (Test-Driven Development) y ATDD (Acceptance Test-Driven Development)		
Criterio	TDD (Test-Driven Development)	ATDD (Acceptance Test-Driven Development)
Significado	Desarrollo guiado por pruebas.	Desarrollo guiado por pruebas de aceptación.
Enfoque principal	Validar el comportamiento técnico del código (unidad o componentes).	Validar que el sistema cumple los requisitos del usuario o negocio.
Nivel de prueba	Pruebas unitarias (nivel de código).	Pruebas de aceptación o funcionales (nivel de sistema).
Quién participa	Principalmente desarrolladores.	Desarrolladores, testers y clientes/usuarios colaboran.
Cuándo se escriben las pruebas	Antes de escribir el código de producción.	Antes de implementar una funcionalidad completa, a partir de criterios de aceptación.
Objetivo	Asegurar que el código funciona constantemente y es mantenible.	Asegurar que la funcionalidad satisface los requisitos del negocio.
Tipo de lenguaje usado en pruebas	Lenguaje de programación (JUnit, NUnit, etc.)	Lenguaje más orientado para negocio (Gherkin, Cucumber, etc.)
Ejemplo típico	"El método suma()" devuelve la suma correcta de dos números.	"Como usuario, quiero poder iniciar sesión para acceder a mi cuenta".
Granularidad	Muy fina (pruebas de métodos o clases).	Más amplia (pruebas de casos de uso o historias de usuario).
Ventaja principal	Mejora la calidad del código y facilita refactorizaciones.	Mejora la comunicación y asegura que se construye lo que el usuario necesita.
Resultado final	Código limpio, probado y libre de errores técnicos.	Funcionalidades completas verificadas con criterios de aceptación.
Herramientas comunes	JUnit, NUnit, xUnit, pytest.	Cucumber, FitNesse, Behave, SpecFlow.
Relación entre ambos	TDD se centra en cómo funciona el código internamente.	ATDD se centra en qué debe hacer el sistema desde la perspectiva del usuario.

Nexus vs. LeSS vs. LeSS Huge			
Framework	Nexus	LeSS	LeSS Huge
Aplicabilidad	De 3 a 9 equipos Scrum.	De 2 a 8 equipos.	Más de 8 equipos.
Miembros y Responsabilidades	1. Nexus Integration Team: Responsable de asegurar un incremento integrado. Compuesto por miembros de los Scrum Teams. 2. Product Owner (PO): Un solo PO para el producto. Responsable de maximizar el valor y gestionar el Product Backlog. 3. Scrum Master (SM): Responsable de que Nexus se entienda. Puede ser SM en uno o más equipos del Nexus. 4. Uno o más miembros del Nexus Integration Team.	1. Product Owner (PO): Un solo PO. 2. Scrum Master (SM): Un SM por línea de 1 a 3 equipos. Es un rol a tiempo completo. 3. Equipos: Autogestionados, multifuncionales. Responsables de coordinar entre ellos.	- Igual a LeSS. Un Product Manager (PO), un DdD, un DdR, Oficiante a LeSS (Mando), 4. Area Product Manager. 5. Area Product Owners.
Eventos	1. Nexus Sprint: Produce un solo incremento integrado. 2. Refinamiento entre equipos: Evento para reducir dependencias e identificar qué equipo hará qué. 3. Nexus Sprint Planning: Coordina actividades de todos los equipos. 4. Nexus Daily Scrum: Asisten representantes de cada equipo. Se centra en problemas de integración. 5. Nexus Sprint Review: Revisamos las revisiones individuales. 6. Nexus Sprint Retrospective: Para planificar mejoras a nivel de Nexus. Se complementa con retros individuales.	1. Sprint Planning One: PO + Equipos (o repa) seleccionan items. 2. Sprint Planning Two: Los equipos deciden cómo harán los items y crean sus Sprint Backlogs. 3. Daily Scrum: Cada equipo tiene la suya. 4. Sprint Review: Una sola review para todo el producto, común a todos los equipos. 5. Retrospective: Cada equipo tiene la suya. 6. Overall Retrospective: Evento nuevo para discutir problemas entre equipos. 7. Product Backlog Refinement: Se realiza con todo el equipo. 8. LeSS Sprint: Un solo Sprint a nivel de producto: todos los equipos empujan y terminan a la vez.	Igual a LeSS. Un Sprint común para todos los equipos.
Artefactos	1. Product Backlog: Un solo Product Backlog. 2. Nexus Sprint Backlog: Compuesto por el Nexus Sprint Goal + Items de los equipos. Compromiso: Nexus Sprint Goal. 3. Integrated Increment: Suma de trabajo integrado. Compromiso: Definition of Done (DoD).	1. Incremento de producto: Salida del Sprint, integra el trabajo de todos los equipos. Un definición de Done (DoD) común para todos los equipos. 2. Product Backlog: Un solo PB para todos los equipos. 3. Sprint Backlog: Cada equipo tiene su propio Sprint Backlog.	- Igual a LeSS. Un Product Backlog para todos, un DdD, un incremento. - Igual a LeSS (Mando), 5. Area Product Backlogs. 6. Area Product Vision.
Filosofía / Gestión	Tradicional (Cascada/Secuencial)	Máticas	Lean (Kanban)
Foco	En los 3 dominios: proceso, proyecto y producto (para mí solo proyecto y producto). Basado en la gestión de proyectos definidos.	En el producto. El "software funcionando" es la métrica principal. Las métricas son para el equipo y no son extrínsecas.	En el proceso. Mide la mejora de un flujo de trabajo continuo.
Métricas claves	1. Esfuerzo -> Proyecto 2. Tiempo -> Proyecto 3. Capacidad -> Producto 4. Defectos -> Producto 5. Costos 6. Riesgos	1. Velocidad (Velocity) -> Producto: La más importante. Mide Puntos de Historia (PF) aceptados por el PO en un Sprint. Se calcula. 2. Capacidad (Capacity) -> Producto: Métrica de proyecto. Mide el compromiso del equipo para un sprint (en horas o SP). Se estima al inicio. 3. Running Tested Features (RTF) -> Producto: Mide la cantidad (absoluta) de features testeadas y funcionando.	1. Lead Time (Tiempo de Entrega) -> Proceso: Perspectiva del cliente. Mide desde que el cliente pide algo hasta que se lo entrega. 2. Cycle Time (Tiempo de Ciclo) -> Proceso: Vista de interna. Mide desde que el equipo empieza a trabajar en algo hasta que lo entrega. 3. Touch Time (Tiempo de Toque) -> Proceso: Mide el tiempo efectivo de trabajo, sin incluir esperas. 4. Efectancia del ciclo de proceso: (Touch Time / Lead Time) -> Proceso.

Testing		
Filosofía / Gestión	Tradicional (Cascada/Secuencial)	Ágil (Iterativo/Incremental)
Momento de pruebas	Las pruebas se realizan al final del ciclo de vida, cuando se entrega el producto.	El testing se realiza por iteraciones.
Otros	(No aplica)	Esta aproximación puede introducir errores por la integración de los sucesivos incrementos?
Diferencias entre Scrum y Kanban		
Diferencia	Scrum	Kanban
Tiempo	Las iteraciones son de tiempo fijo.	El tiempo fijo es opcional. La cadencia puede variar. Pueden estar marcadas por la previsión de los eventos en lugar de tener un tiempo prefijado.
Compromiso	El equipo asume un compromiso de trabajo por iteración.	El compromiso es opcional.
Métrica por defecto	Para la planificación y mejora es la velocidad.	Por defecto es el Lead Time (tiempo de entrega promedio).
Reglas	Multifuncionales.	Multifuncionales o especializadas.
Diagramas de seguimiento	Deben emplearse gráficos Burndown.	No se prescriben diagramas de seguimiento.
Limitación WIP (Work in Progress)	Es indirecta y está marcada por el sprint.	Es directa para cada etapa de trabajo o estado del trabajo.
Agrupar trabajo	No se puede agrupar alcance una vez comenzado el sprint.	Siempre que haya capacidad disponible, se puede agrupar trabajo.
Compartido entre equipos	Cada equipo tiene su sprint backlog.	La estimación es opcional.
Permanencia del tablero	En cada sprint se limpia.	Todos los equipos comparten la misma pizarra o tabla Kanban.
Roles	Se prescriben tres roles: PO (Product Owner), SM (Scrum Master) y Equipo.	El tablero es persistente.
Priorización	El product backlog debe estar priorizado.	No se prescriben roles.
Funcionalidades	Se espera que las funcionalidades se dividan tal que se completen en un sprint.	La priorización es opcional.
		No se prescribe el tamaño de la funcionalidad.

Análisis de los principios Lean y explicación de las prácticas que propone Kanban para aplicar en forma concreta los principios	
1. Eliminar desperdicios	Visualizar el trabajo: permite visualizar el flujo de trabajo y exponer aquellas actividades que no aportan valor y aquellas que generan tiempos de espera que "traban" el flujo de trabajo.
2. Amplificar el aprendizaje	Limitar el WIP: el trabajo a medias genera retrabajo, lo cual es un desperdicio. Al limitar el WIP se evita ese trabajo a medias para concentrar esfuerzos.
3. Hacer explícitas las políticas	Hacer explícitas las políticas: que todos los miembros tengan acceso a las políticas usadas para cada unidad de trabajo, actividades a realizar, información necesaria del dominio, etc.
4. Posponer decisiones hasta el momento responsable	Evaluación y mejora continua: no introducir cambios grandes, sino que visualizar el proceso de trabajo actual y aplicar mejoras en él.
5. Ver el todo	Embeber la integridad conceptual
6. Dar poder al equipo	Limitar el WIP: particularmente el WIP en la columna backlog.
7. Mejorar colaborativamente	Visualizar el trabajo: a través del tablero permite que todos los miembros observen todo el trabajo que se hace, el sentido de cada una de las tareas y cómo estas conforman el flujo de trabajo como un todo que es esencial para la satisfacción de los clientes.
8. Entregar lo antes posible	6. Dar poder al equipo
	Haciendo explícitas las políticas y conformando un sistema de trabajo pull, donde cada miembro suma el trabajo cuando esté en condiciones de realizarlo, y no se lo impone nadie.
	7. Mejorar colaborativamente
	8. Entregar lo antes posible
	Gestionar el flujo de trabajo: permite ver de principio a fin los pasos que se deben seguir para lograr cumplir con las necesidades del cliente y hacer aquellos ajustes que sean necesarios para entregar el mayor valor posible, en el menor tiempo posible.

Cuadro comparativo Auditorías vs. Revisiones Técnicas		
Aspecto	Auditoría	Revisión
Para qué sirven	Para revisar qué se está haciendo en la realidad, comparado con lo que se le propuso a hacer.	Revisar errores en un análisis lo antes posible, para estar lo más preparados a hacer.
Quién	Auditor: auditado y gerente de calidad.	Auditor, moderador, auditado, lector, inspector.
Beneficios	Opiniones independientes, identificar áreas de mejoramiento, asegurar que se cumplen expectativas, dar visibilidad a procesos de trabajo, etc.	Detección de errores, evitar el desperdicio de recursos, asegurar que se cumplen expectativas, dar visibilidad a procesos de trabajo, etc.
Etapas	1. Planificación 2. Desarrollo 3. Análisis y Recuento de resultados 4. Seguimiento	En revisiones informales no hay pasos a seguir. En revisiones formales: 1. Planificación 2. Visión general 3. Revisión 4. Revisión de especificación 5. Corrección 6. Seguimiento
Tipo	De proyecto, de calidad, de producto (física o funcional).	Formales e informales.

100

Cuadro comparativo CMMI por Etapas y Continuo		
Aspecto	Por etapas	Continuo
Características	Análisis determinado nivel de madurez de una organización como un todo, dependiendo de qué tanto creemos madurez, de acuerdo a la especificación en el modelo CMMI.	Análisis determinado nivel de madurez de un área de proceso de una organización, dependiendo de qué tanto creemos madurez, considerando a esa área, de acuerdo a lo que la organización quiere.
Ventajas	Permite integrar las formas de trabajo de toda la organización. Identifica cómo nivel de calidad en toda la organización, según el nivel que se quiere. Permite saber bases en la organización y entender los niveles de calidad en toda la empresa.	Permite a una organización especializarse en un área de proceso, independientemente del resto de áreas. La organización sigue sus prácticas reales. De acuerdo al área en la que se especializa.
Desventajas	Se deben realizar ciertos actividades en áreas de proceso que quizás no sean relevantes para la empresa. La organización como un todo se sigue considerando, independientemente de que no requieren las actividades que plantea CMMI nivel 1 en las áreas de proceso.	Puede quedar áreas de proceso sin demasiada calidad, debido a que se centran los esfuerzos en una zona. La organización como un todo se sigue considerando, independientemente de que no requieren las actividades que plantea CMMI nivel 1 en las áreas de proceso.
Diferencias	- Modelo 1 a 5 - Niveles: indican madurez organizacional - Definidos por un conjunto de áreas de proceso - Permite una secuencia para mejorar determinadas áreas de proceso	- Modelo 0 a 5 - Niveles: indican capacidad de un área de proceso - Definidos por cada área de proceso - Permite mejorar en un área de proceso que se desea

hallazgos	Buenas prácticas, observaciones	Errores

Cuadro comparativo Gestión Tradicional vs. Gestión Ágil		
Aspecto a comparar	Gestión Tradicional	Gestión Ágil
Requisitos a cambios	Poco flexibles	Altamente flexibles
Requisitos en el desarrollo del proyecto	Agg -> se define en etapas de desarrollo	Agg -> se define cuando sea necesario
Tiempo de entrega de software (SI)	Variable	Temperano
Retransmisión del cliente para incluir nuevos requerimientos	Alto	Bajo
Participaciones en estimaciones	Un hace el líder de proyecto	Los hace el equipo de desarrolladores
Confirmación de equipo	Indirecta, con roles definidos	Autogestionado según fortalezas de cada miembro
Proceso de desarrollo	Definido	Implicado

100

Códo de vida admitido	Cualquiera	Restricto e incremental
Se estima	Recursos y tiempos necesarios para el desarrollo	Alumnos a implementar en iteraciones (Cascada)
Se comienta	Los requisitos identificados en etapas de requerimientos	El equipo, recursos y tiempo de trabajo
Planificación de tareas	Estimados con detalle al planificar	Estimados con detalle al planificar
Preguntas frecuentes de la ingeniería de software (Sumarias)		
¿Cuál es software?	Programas de cómputo y documentación asociados. Los productos de software se desarrollan para su cliente en particular y para un mercado en general.	
¿Cuáles son los atributos de buen software?	El buen software debe entregar al usuario la funcionalidad y el desempeño requeridos, y debe ser sustentable, confiable y adaptable.	
¿Cuál es ingeniería de software?	La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.	
¿Cuáles son las actividades fundamentales de la ingeniería de software?	Especificación, desarrollo, verificación y evaluación del software.	
¿Cuáles son las principales metas que enfrenta la ingeniería de software?	Se enfrentan con una cantidad creciente, demandando por tiempos de desarrollo más cortos y desarrollo de software confiable.	

100

¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?	Aun cuando todos los proyectos de software deben gestionarse y desarrollarse de manera profesional, existen diferencias de manera adecuada para distintos tipos de sistema. Por ejemplo, los juegos comerciales deben cumplir con una serie de requisitos, mientras que los sistemas críticos de medicina de seguridad requieren de una especificación completa y detallada para su desarrollo. Por lo tanto, no puede decirse que un método sea mejor que otro.	